

1. Executive Summary

Native Charm Alpha Vaults V2 is a managed Uniswap v3 LP strategy with strong contract hygiene but material economic dependence on rebalance policy, manager quality, and market microstructure. The contracts do what they advertise (deposit/withdraw/rebalance with explicit guards and fee accounting), but economic outcomes remain path-dependent in trend and gap regimes, where adverse selection can dominate fee income.

4626fun does not simply expose native Charm shares. It adds a multi-layer integration (CreatorCharmStrategy -> CreatorOVault ERC-4626 -> CreatorOVaultWrapper + ShareOFT normalization), which introduces additional valuation and redemption semantics risk. This layering increases the gap between accounting value and executable exit value, especially under stale oracle/TWAP state or constrained unwind conditions.

The most dangerous accounting misconception is equating `getTotalAmounts()` or wrapped `totalAssets()` with guaranteed synchronous redemption value. Both stacks expose accounting views that can diverge from immediate realizable exits due to stale fee realization, swap path availability, queueing constraints, and live market impact.

The most dangerous market-structure risk is predictable rebalance behavior around concentrated ranges. Native Charm rebalance windows and 4626fun automation triggers create opportunities for sophisticated flow to extract value around LP inventory transitions without violating any contract-level invariant.

2. Repository Map

Repo	Files reviewed	Purpose	Audit relevance
charmfinance/alpha-vaults-v2-contracts	README.md, contracts/AlphaProVault.sol, contracts/AlphaProVaultFactory.sol, contracts/AlphaProPeriphery.sol, contracts/ManagerStore.sol, interfaces/IVault.sol, test/deposit-withdraw.test.ts, test/governance-methods.test.ts, test/invariants.test.ts, test/periphery.test.ts, test/rebalance.test.ts, test/helpers.ts, test/factory.test.ts, deploy/00_deploy_charm.ts, deploy/01_deploy_mock.ts, deploy/02_deploy_manager.ts, deploy/03_deploy_periphery.ts, utils/constants.ts, package.json, hardhat.config.ts	Native strategy logic, governance, valuation helpers, tests, deployment assumptions	Source of truth for inherited Charm behavior/risk

Repo	Files reviewed	Purpose	Audit relevance
4626fun/contracts	contracts/vault/Creator0Vault.sol, contracts/vault/modules/Creator0VaultCoreModule.sol, contracts/vault/modules/Creator0VaultStrategiesModule.sol, contracts/vault/modules/Creator0VaultAdminModule.sol, contracts/vault/Creator0VaultWrapper.sol, contracts/vault/strategies/univ3/CreatorCharmStrategy.sol, contracts/interfaces/IStrategy.sol, contracts/interfaces/IStrategyValuation.sol, contracts/interfaces/ICharmFactory.sol, contracts/helpers/batchers/StrategyDeploymentBatcher.sol, contracts/helpers/batchers/DeploymentBatcher.sol	ERC-4626 integration, strategy adaptation, queueing, valuation guards, deployment-time policy	Source of integration-specific risk
4626fun/frontend + 4626fun/cre	frontend/src/pages/Vault.tsx, frontend/src/pages/Status.tsx, frontend/api/_handlers/status/_vaultReport.ts, frontend/api/_handlers/v1/charm/_strategy.ts, frontend/server/keeprxmtQueueExecutor.ts, frontend/server/_lib/privyCoinbaseSmartWallet.ts, frontend/server/_lib/privyXmtSigner.ts, frontend/server/_lib/keeprSchema.ts, frontend/server/_lib/keeprRegistry.ts, frontend/server/_lib/keeprAutomation.ts, cre/workflows/4626.workflow.ts, cre/workflows/charm-rebalance-manager.workflow.ts, cre/workflows/strategy-event-listener.workflow.ts, cre/workflows/keepr-queue-executor.workflow.ts, cre/actions/charm-rebalance-manager.action.ts, cre/actions/keepr-queue-executor.action.ts, cre/actions/strategy-event-listener.action.ts, cre/cre-workflows/_shared/charmManager.ts, cre/utils/onchain.ts, cre/config.ts	Operator/automation trust boundary, queue liveness, CSW signing path, user-facing valuation display	Source of operator and liveness risk

3. System Architecture

- Native Charm onchain architecture
- AlphaProVault manages three-order CLMM state (full/base/limit), fee accrual state, and rebalance parameters.
- Uniswap v3 pool state is external truth; vault stores policy and accounting overlays.

- AlphaProVaultFactory deploys clone vaults and owns protocol-fee governance.
- AlphaProPeriphery reconstructs position/fee views for monitoring.
- Offchain / manager / delegate architecture
- Rebalance timing is manager/delegate-driven (or open to anyone before delegate assignment).
- Parameter updates (baseThreshold, limitThreshold, period, maxTwapDeviation, fees) are discretionary manager/governance policy.
- Fee changes are applied at rebalance boundaries, not instantly at setter call.
- 4626fun integration architecture
- User surface: CreatorOVaultWrapper (deposit/withdraw) wrapping CreatorOVault shares into ShareOFT.
- Vault core: CreatorOVault is ERC-4626 with strategy debt tracking, valuation readiness gating, withdraw delay, and large-withdraw queue.
- Strategy layer: CreatorCharmStrategy holds Charm shares + idle CREATOR/USDC, values USDC via CreatorOracle, and uses V3 TWAP for swap sizing/slippage checks.
- Deployment layer: DeploymentBatcher deploys official Charm vault + strategy adapters and sets ownership/manager roles.
- Optional automation / CRE architecture
- Multiple workflows can influence charm rebalance behavior: direct charm-rebalance-manager, event-driven enqueue (strategy-event-listener / charmManager), and queue executor (keeper-queue-executor).
- CSW/ERC-4337 path resolves owner index onchain and submits UserOps via bundler/paymaster.
- Queue state machine (pending -> executing -> executed | retry | failed) provides retries/backoff with dedupe keys.

4. Critical Assumptions

Assumption	Where it lives	Why it matters	Failure mode if false
Rebalance policy is economically sane	AlphaProVault.rebalance + manager setters	Core PnL driver in CLMM	Value bleed without contract bug
getTotalAmounts is accounting, not guaranteed executable exit	AlphaProVault.getTotalAmounts, CreatorCharmStrategy.getTotalAssets	Prevents NAV overconfidence	Mispriced mint/redeem behavior
TWAP guard parameters are practical	checkCanRebalance, batcher defaults	Liveness + manipulation resistance	Freeze or churn
Oracle freshness reflects usable conversion conditions	CreatorOracle, CreatorCharmStrategy.isValuationReady	USDC leg valuation and gatekeeping	Stale-NAV or deposit halt

Assumption	Where it lives	Why it matters	Failure mode if false
USDC -> CREATOR unwind path is usually available	CreatorCharmStrategy.withdraw	CREATOR-denominated withdrawals depend on it	Partial withdrawals, stranded USDC
Queue and executor remain live	keepr_actions + CRE queue workflows	Strategy/admin actions must be consumed	Operational liveness failure
CSW owner index remains resolvable	privyCoinbaseSmartWallet helpers	ERC-4337 execution requires valid owner slot	Silent keeper failure after owner drift
Dedupe keys are used consistently by all producers	enqueueKeeprAction, charm manager/event listener	Prevent duplicate action execution	Double-trigger churn/race
UI distinguishes accounting vs realizable values	Vault.tsx, _vaultReport.ts	Prevents user semantic mismatch	False redemption expectations
Wrapper flow includes equivalent queue semantics	Creator0VaultWrapper	Large exits require queue path in vault core	Wrapper users hit revert-only path for large exits

5. Native Charm Contract Review

- AlphaProVault
- deposit calls _poke on all positions before share math (_calcSharesAndAmounts), then mints shares and checks cap.
- withdraw burns proportional liquidity from all three positions and transfers proportional fees net fee skims.
- rebalance burns all current positions, computes fresh ranges from current tick and thresholds, then places full-range, base-range, and one-sided residual order.
- checkCanRebalance enforces period, tick movement, TWAP deviation, and boundary safety checks.
- protocolFee and managerFee are applied in _burnAndCollect; active fee values are rolled at rebalance (protocolFee = factory.protocolFee(), managerFee = pendingManagerFee).
- Factory
- createVault is public and deploys minimal-proxy clones.
- Governance controls protocolFee and governance handoff (setGovernance/acceptGovernance).
- Native protocol-fee authority is centralized in governance, not per-vault manager.
- Periphery
- getVaultPositions + getFees + calculateFees reconstruct position and fee views from pool state.
- Good for observability; not a liquidation oracle or guaranteed execution-value estimator.
- ManagerStore

- Registry/authorization utility (registerManager, authorizeManager) and not the direct auth gate in vault execution.
- Interface assumptions (IVault)
- Exposes many read surfaces that can look authoritative (getTotalAmounts, fee states, ranges).
- Integrators can incorrectly treat these as execution-guaranteed values if semantics are not made explicit.

6. Accounting and Valuation Review

Deposit / Share / Withdrawal mechanics (Q6-Q16):

- Q6: Shares are minted/burned pro-rata against vault holdings. First deposit permanently locks MINIMUM_LIQUIDITY.
- Q7: A share is a claim on idle balances + CLMM position inventory + claimable fees net protocol/manager fee policy.
- Q8: Deposits can sit idle until rebalance; immediate productive deployment is not guaranteed.
- Q9: Native minting is relatively fair because deposit pokes positions first; however fairness still depends on timing and live market state.
- Q10: getTotalAmounts() is an accounting estimate, not guaranteed synchronous withdraw output.
- Q11: getPositionAmounts() uses liquidity math + owed tokens; it does not model market impact, path liquidity, or execution timing.
- Q12: Yes, fees since last poke can create stale accounting windows.
- Q13: Tiny deposits that trigger poke can alter fee-capture timing between cohorts.
- Q14: Early/late entrants can be advantaged depending on poke/rebalance cadence and market trajectory.
- Q15: First deposit, locked minimum liquidity, and rounding are meaningful edge surfaces.
- Q16: Yes, users can be misled if wrappers present previews as equivalent to immediate executable exits.

Periphery / total-assets semantics (Q39-Q50):

- Q39: AlphaProPeriphery is a monitoring helper for positions and fee reconstruction.
- Q40: Safe for monitoring; unsafe as sole source for mint/redeem fairness decisions.
- Q41: Under stress, it can diverge from true realizable value because it has no execution/slippage model.
- Q42: Directionally consistent with vault fee accounting, but still assumption-bound.
- Q43: Drift sources include unpoked fees, stale conditions, out-of-range composition, and fee update boundaries.
- Q44: If presenting one NAV number, use a hybrid model: conservative accounting + freshness flags + executable-liquidity context.
- Q45: Wrong approach is pure spot-marked accounting with no stale/liquidity haircuts.
- Q46: In 4626 wrapping, totalAssets() should represent conservative redeemable-equivalent value, not optimistic mark.
- Q47: Safest is a hybrid (TWAP/oracle + safety haircut + freshness gating + liveness signal).
- Q48: Safest deposit valuation is conservative/lower-bound (prevents cheap-share minting).
- Q49: Safest withdrawal valuation is queue-aware executable estimate (prevents over-promising).
- Q50: Wrong valuation model can create silent unfair-share transfers and practical insolvency-style redemption failures.

7. Concentrated Liquidity Strategy Review

- Q17: Capital allocation sequence is full-range, then base, then one-sided residual bid/ask.
- Q18: Strongest in mean-reverting fee-rich markets with disciplined rebalance timing.
- Q19: Toxic in sustained trends, sharp gaps, and low-depth conditions where adverse selection dominates.
- Q20:
 - trending: inventory drifts and IL/adverse selection rise,
 - sharp gaps: ranges stale quickly,
 - mean reversion: fee capture improves,
 - volatility spikes: fee opportunity plus extraction risk,
 - prolonged one-sided flow: residual order can magnify directional inventory.
- Q21: Yes, one-sided residual order can amplify risk.
- Q22: Exposure exists to inventory imbalance, adverse selection, informed flow, and timing games around rebalance.
- Q23: Yes, fees can look strong while net inventory damage dominates.
- Q24: Full-range slice is a tail hedge trade-off; with `fullRangeWeight=0` (4626 deployment defaults), that hedge is intentionally removed.
- Q25: Yes, threshold choices can be structurally fragile. Current deployment defaults (`base=3000`, `limit=6000`, `period=1800`, `maxTwapDeviation=0`, `twapDuration=60`) are especially liveness-sensitive.

8. Rebalance and MEV Review

- Q26: rebalance burns all liquidity, snapshots, computes new ranges from current tick, mints full/base/residual orders, then updates last tick/time and active fee parameters.
- Q27: Access controls are conditional and not uniformly strict over lifecycle.
- Q28: Before `rebalanceDelegate` is set, anyone can call rebalance.
- Q29: This can be situationally acceptable for permissionless operation but dangerous under weak guard config or hostile microstructure.
- Q30: Preconditions (`period`, `minTickMove`, `maxTwapDeviation`, boundary checks) are explicit but only as good as parameterization.
- Q31: Attackers can nudge pass/fail near thresholds in low-depth windows.
- Q32: Manager/delegate can still execute economically poor but rule-compliant rebalances.
- Q33: Deterministic eligibility windows can be predicted and targeted.
- Q34: Searchers can extract around rebalance submission and new range placement.
- Q35: TWAP protection helps against very short spikes but is not a comprehensive anti-MEV layer.
- Q36: Stale TWAP or low-liquidity contexts can make checks economically misleading.
- Q37: There are states where rebalance is code-permitted but economically suboptimal (thin books after large move).
- Q38: There are states where rebalance is economically needed but code can block it (e.g., strict `maxTwapDeviation=0`).

9. Governance and Operator Risk Review

- Q51 (manager privileges): threshold updates, full-range weight, period/tick guards, twap duration, max supply, manager fee (pending), manager handoff, rebalance delegate, fee collection, sweep (non-token0/1), emergency burn, and effective rebalance control.
- Q52 (factory governance privileges): protocol fee updates, governance handoff, and protocol fee collection authority.
- Q53 (rebalance delegate): can execute rebalance but not broader policy setters.
- Q54 (emergencyBurn): can force liquidity removal from chosen ranges; operationally powerful and economically sensitive.
- Q55 (sweep): comparatively well scoped because token0/token1 are blocked.
- Q56: Threshold/cap/fee changes can be used irresponsibly or maliciously without violating onchain invariants.
- Q57: Fee changes activating on rebalance create timing edge cases around user flows.
- Q58: Two-step handoffs reduce fat-finger risk but introduce governance-latency risk windows.
- Q59: Yes, manager role is economically equivalent to active fund operator control.
- Q60 (required user disclosures in 4626fun): active management dependence, mutable policy risk, non-guaranteed synchronous exits, accounting-vs-realizable value gap, and automation/oracle dependency.

10. Test Coverage Review

- Q61 (covered well)
- Native: deposit/withdraw happy paths, role gates, governance methods, rebalance guard checks, periphery consistency.
- 4626fun: valuation readiness gating, strategy-valuation fallback behavior, oracle freshness semantics, transfer exactness, cooldown anti-bypass, wrapper normalization/dust accounting, ShareOFT behavior validation.
- Q62 (major misses)
- End-to-end stressed exits with explicit market impact.
- Adversarial MEV simulation around rebalance windows.
- Full automation outage/race/duplicate execution under production-like failure modes.
- Q63 (missing tests by category)
- stressed exits: largely missing,
- gap moves: missing,
- adverse selection: missing,
- public rebalance abuse: partially acknowledged, not adversarially modeled,
- stale-fee fairness divergence: limited,
- vault/periphery divergence under stress: limited,
- malicious manager behavior: limited,
- delayed/broken automation: missing E2E coverage.
- Q64 (10 tests to add first) 1) Native Charm rebalance sandwich simulation around eligibility edge. 2) Trend-path net PnL decomposition (fees vs inventory damage). 3) Low-liquidity TWAP threshold manipulation boundary tests. 4) 4626 stale-poke mint fairness differential test. 5) Wrapper large-withdraw queue-path equivalence test. 6) Queue claim-value drift test over delay window. 7) Persistent USDC unwind failure impact on vault withdrawal liveness. 8) Queue dedupe race across overlapping workflow producers. 9) CSW owner-index drift and recovery path test. 10) Bundler/paymaster outage retry/backoff/dead-letter behavior test.

11. 4626fun Integration Review

- Q65: 4626fun is not directly exposing native AlphaProVault shares to users. It deploys official Charm vaults, adapts them through CreatorCharmStrategy, wraps inside ERC-4626 CreatorOVault, and then wraps again through CreatorOVaultWrapper + ShareOFT.
- Q66: inherited vs newly created risk per layer
- Charm inherited: CLMM inventory/adverse-selection risk, rebalance policy dependence.
- New in 4626:
 - oracle-valued USDC leg in CREATOR accounting,
 - ERC-4626 preview semantics atop path-dependent exits,
 - normalization/dust accounting layer in wrapper,
 - queue and automation liveness as explicit trust boundary.
- Q67: semantic mismatch exists between ERC-4626 concepts (deposit, mint, previews, totalAssets) and concentrated-liquidity realizability.
- Q68: yes, synchronous wrapper UX can accidentally imply immediate redemption where underlying path is slippage/queue dependent.
- Q69: yes, displayed share/NAV smoothness can mask unstable underlying inventory composition.
- Q70: yes, stale NAV windows can allow cohort unfairness (new users arbing old users).
- Q71: batching can reduce some timing games, but can worsen fairness if the batch valuation input is stale/optimistic.
- Q72: async/queued rebalances move risk from immediate executors to queued users and remaining holders during delay windows.

12. Automation / CRE Review

- Q73: current behavior is hybrid: direct rebalance manager, queue-based execution, event-listener enqueueing, plus consolidated workflow.
- Q74: deterministic/onchain should include auth checks, hard eligibility constraints, and mint/redeem accounting invariants.
- Q75: safe offchain automation scope includes monitor/queue/validate/simulate and deterministic relay.
- Q76: AI must never be authority for auth decisions, threshold updates, fee updates, or emergency execution.
- Q77: recommended CRE role is monitor + queue + validate + simulate + gate; direct execution only as deterministic, policy-constrained relay.
- Q78: CRE improves observability/guardrails but current multi-path topology adds operational complexity.
- Q79: added failure modes include duplicate producers, stale queue, owner-index drift, bundler/paymaster outages, API/db outages.
- Q80: minimal safe CRE role is a single canonical action producer plus a single canonical executor with strict idempotency and replay controls.

13. Attack Scenarios

Scenario	Preconditions	Exploit path	Impact	Detectability	Mitigation	Severity
81. public rebalance exploitation before delegate is set	rebalanceDelegate=0, permissive guard params	External actor times rebalance at adverse moment	Value loss / unfair transfer	Medium	Set delegate immediately; tighten guard params	High
82. manager-triggered bad rebalance	Manager compromise/error	Rule-compliant but economically poor range reset	Value loss	Medium	Timelock + policy limits + monitoring	High
83. rebalance front-running/back-running	Predictable rebalance eligibility	Searchers position around tx and post-range state	MEV extraction from LP	Medium	Private relay + jitter + stronger guards	High
84. deposit before rebalance to capture stale accounting	Stale valuation window	Deposit while NAV lags true state	Cohort unfairness	Low/Medium	Conservative mint valuation	High
85. withdraw after favorable poke before unfavorable rebalance	Timing visibility	Exit after fee/accounting refresh	Unfair transfer to remainers	Medium	Queue/anti-timing policy	Medium
86. poke-induced fee capture timing game	Poke windows externally triggerable	Tiny tx causes fee-state refresh timing edge	Cohort unfairness	Medium	More explicit fee realization policy	Medium
87. stale valuation in 4626 wrapper	Oracle stale or conversion unavailable	Wrapper uses stale optimistic accounting	Unfair share transfer / redemption mismatch	Medium	Freshness hard gates + valuation haircut	Critical
88. mispriced ERC-4626 share issuance	Optimistic totalAssets vs constrained exits	Mint/redemption priced off non-executable value	Silent unfairness / insolvency tendency	Low until stress	Pessimistic executable valuation	Critical
89. one-sided residual order exploitation	Directional flow near rebalance	Trade against predictable residual side	Inventory loss	Medium	Adaptive residual policy	High
90. adverse selection during trend moves	Persistent trend	LP inventory repeatedly selected against	Net value bleed	High	Faster adaptive rebalance + tighter risk controls	High
91. TWAP guard bypass/insufficiency	Short windows/low depth	Push tick around threshold conditions	Wrong rebalance decisions	Medium	Longer windows + liquidity gating	High

Scenario	Preconditions	Exploit path	Impact	Detectability	Mitigation	Severity
92. forced rebalance near boundary conditions	Boundary-adjacent tick	Rebalance while structurally fragile	Value loss / liveness risk	Medium	Additional volatility/impact checks	High
93. manager fee change timing exploitation	Pending manager fee	Activate near favorable/unfavorable flow	User fairness issue	High	Notice windows + timelock	Medium
94. protocol fee change timing exploitation	Pending protocol fee	Governance times fee switch around flows	User fairness issue	High	Timelock + change disclosure	Medium
95. emergency burn misuse or panic harm	Emergency authority misuse	Force liquidity unwind at poor time	Value loss + liveness disruption	Medium	Restricted emergency runbook	High
96. offchain automation outage	Queue/keeper path down	Rebalances/reports /actions stall	Liveness failure -> economic drift	High	Redundant executor + oncall runbook	High
97. CRE double-trigger / duplicate action	Multiple producers + retries	Duplicate rebalance actions enqueued/executed	Churn/gas/possible MEV drag	Medium	Single producer + strict dedupe keys	Medium
98. fake/optimistic NAV reporting to users	UI presents accounting as executable	Users act on misleading value semantics	Trust + fairness harm	Medium	Show accounting vs executable values separately	High

14. Stress Test Outcomes

Stress scenario	Accounting effect	Real exit effect	User impact	Recovery path	Severity
99. 5% move within range	Minor accounting drift	Usually manageable	Limited	Automatic + normal rebalance cadence	Medium
100. 15% trend through range	Accounting can lag inventory reality	Exit quality degrades	Early-exit advantage	Manager/keeper-dependent rebalance	High
101. 30% sharp move with thin liquidity	Accounting may remain optimistic briefly	Severe slippage/unwind constraints	Late users penalized	Manager intervention + cautious unwinds	Critical
102. Volatility spike with whipsaw	Frequent accounting oscillation	Rebalance churn and extraction risk	Performance unpredictability	Policy tuning + guarded cadence	High

Stress scenario	Accounting effect	Real exit effect	User impact	Recovery path	Severity
103. 50% TVL withdraws after large move	NAV may still look intact	Buffer/unwind bottleneck, potential queue pressure	Reverts/delays for many users	Queue + emergency operations	Critical
104. Rebalance delayed 6h / 24h / 72h	Increasing mark-vs-realization gap	Stale positioning and adverse selection	Progressive fairness erosion	Restored automation + manual override	High/Critical
105. Heavy fees accrue but no poke	Accounting undercounts fees until refresh	Real withdraw can differ from stale view	Timing-sensitive fairness	Any poke/rebalance refreshes	Medium
106. Aggressive threshold changes mid-operation	Regime shift in accounting expectations	More fragile rebalance behavior	User predictability loss	Governance rollback and retune	High
107. Fee changes just before flows	Accounting remains coherent but economics shift	Net user outcomes discontinuous	Fairness/trust concern	Timelock + transparent notice	Medium
108. Wrapper uses stale valuation	Smooth displayed NAV	Exit mismatch or failed expectations	Hidden risk transfer	Strict freshness gating	Critical
109. Automation/CRE does not fire	Aging accounting and stale monitoring	No corrective policy execution	Liveness failure	Runbook/manual ops required	High
110. Automation/CRE fires twice	Accounting mostly unchanged	Churn and execution-noise losses	Operational instability	Dedupe and idempotent sink logic	Medium
111. Searchers target predictable rebalance windows	Deterministic trigger windows	Persistent extraction around resets	Long-run yield drag	Private relay + jitter + trigger hardening	High

15. Findings

Critical

- title: ERC-4626 accounting can diverge from realizable Charm exit value severity: Critical affected component: Creator0Vault, CreatorCharmStrategy, Creator0VaultWrapper evidence: Creator0Vault.totalAssets, CreatorCharmStrategy.getTotalAssets, wrapper synchronous withdraw path why it matters: share mint/redeem fairness can diverge from executable exits exploitability: timing + stale/liquidity-constrained states recommended fix: use pessimistic executable valuation and expose queue-aware redemption semantics

- title: Wrapper lacks full queue-equivalent large redemption path severity: Critical affected component: `CreatorOVaultWrapper` evidence: `wrapperWithdraw` always calls synchronous `vault.redeem`; queue functions are in vault core only why it matters: large exits can revert while wrapper UX implies one-step redemption exploitability: stress scenarios and large holders recommended fix: add wrapper queue/claim/cancel interfaces or enforce async redemption product-wide
- title: Strategy USDC unwind failure can leave accounting-exposed but non-returned value severity: Critical affected component: `CreatorCharmStrategy.withdraw` evidence: `withdraw` tolerates swap failure and returns only available `CREATOR` while USDC can remain idle why it matters: user withdrawal expectations may exceed realizable immediate output exploitability: low-depth / unavailable TWAP or swap path recommended fix: strict unwind requirement for user redemptions or explicit queued settlement model

High

- title: Public rebalance access before delegate assignment severity: High affected component: `AlphaProVault.rebalance` evidence: `manager/delegate` check only enforced when `rebalanceDelegate != 0` why it matters: outsiders can force economically adverse resets exploitability: practical in live markets recommended fix: set delegate at deployment and block rebalance until explicit assignment
- title: Deployment defaults create brittle guard regime severity: High affected component: `DeploymentBatcher` Charm parameters evidence: `minTickMove=0`, `maxTwapDeviation=0`, `twapDuration=60` in strategy deployment path why it matters: can freeze needed rebalances or create fragile trigger behavior exploitability: market-structure dependent recommended fix: adjust defaults and enforce validated parameter envelope
- title: Manager/governance discretion is fund-operator grade without enforced delay controls severity: High affected component: `AlphaProVault` + `AlphaProVaultFactory` evidence: live setters for thresholds, fees, caps, delegates, manager/governance handoffs why it matters: user outcomes heavily depend on privileged operator behavior exploitability: malicious or accidental misuse recommended fix: timelock, bounded parameter changes, explicit disclosure
- title: Multi-path charm automation increases duplicate/race surface severity: High affected component: CRE workflows + queue execution evidence: direct rebalance workflow + event-driven enqueue + queue-based execution all present why it matters: action churn, retry storms, and inconsistent operational behavior exploitability: mostly accidental but amplifiable recommended fix: one canonical producer and one canonical executor per strategy action domain

Medium

- title: Periphery values are easy to over-trust as executable NAV severity: Medium affected component: `AlphaProPeriphery`, reporting endpoints evidence: model-based fee/position reconstruction without execution impact model why it matters: can mislead operators/users on liquidation-time value exploitability: decision-quality degradation recommended fix: mark as monitoring-only and pair with executable-liquidity indicators
- title: Fee realization timing can shift value between cohorts severity: Medium affected component: Charm poke/rebalance lifecycle evidence: accounting updates happen on poke/rebalance boundaries why it matters: timing-savvy users can capture relative advantage exploitability: moderate recommended fix: more deterministic refresh cadence and anti-timing batching

- title: Queue claim value is inherently path-dependent across delay window severity: Medium affected component: `CreatorOVault` queue withdrawal mechanics evidence: queued shares redeem at claim-time conditions why it matters: queued users bear market drift during unlock delay exploitability: market dependent recommended fix: explicit UX disclosure + optional claim-time deviation guard

Low

- title: High first-deposit minimum centralizes bootstrap path severity: Low affected component: `CreatorOVault` first deposit guard evidence: `MINIMUM_FIRST_DEPOSIT = 50_000_000e18` why it matters: operational bottleneck and reduced decentralization at launch exploitability: low recommended fix: controlled bootstrap runbook and post-launch configurable threshold policy
- title: Strategy interface `rebalance()` semantics differ from actual Charm rebalance path severity: Low affected component: `CreatorCharmStrategy.rebalance` vs executor behavior evidence: strategy `rebalance()` emits event but does not call `charmVault.rebalance`; executors call Charm vault directly why it matters: integration assumptions can be wrong for downstream callers exploitability: low direct exploit, medium integration confusion risk recommended fix: either implement true pass-through rebalance or rename/document semantics explicitly

Informational

- title: `ManagerStore` is auxiliary and not native vault auth root severity: Informational affected component: Charm manager registry stack evidence: vault role checks are local (on `LyManager`) why it matters: prevents misplaced trust in registry metadata exploitability: n/a recommended fix: document role separation clearly
- title: Native Charm operational docs are minimal severity: Informational affected component: Charm README/deployment docs evidence: limited production runbook guidance for rebalance policy why it matters: higher operator variance in integration behavior exploitability: indirect recommended fix: maintain a protocol-level risk policy and deployment checklist

16. Recommended 4626fun Architecture

Chosen option: **C. Use it only behind an async / queued redemption model.**

Justification:

- The underlying strategy is concentrated-liquidity and path-dependent by design.
- Integration adds additional layers (`CreatorCharmStrategy`, `CreatorOVault`, wrapper normalization) that increase semantic mismatch risk for synchronous UX.
- Queue-aware redemption best matches true executable risk and avoids over-promising instant exits.

Implementation direction:

- Keep accounting and execution values separate in all APIs/UIs.
- Gate mint/deposit on strict valuation freshness and bounded deviation checks.
- Route large redemptions through explicit queue lifecycle at the wrapper level.
- Collapse charm automation into a single canonical action producer/executor path.

17. Production Readiness Verdict

Scores (1-10):

- native contract safety: **7**

- accounting integrity: **5**
- valuation honesty: **4**
- rebalance robustness: **4**
- concentrated-liquidity risk tolerance: **5**
- governance/operator safety: **5**
- 4626fun integration suitability: **4**
- automation/CRE suitability: **5**
- overall deployability: **4.5**

Would you deploy this with real user funds today?

- **No**, not with current synchronous wrapper semantics and current valuation/automation coupling.

Exact blockers to resolve first:

- 1 Queue-equivalent redemption path in wrapper UX/contracts.
- 2 Conservative executable valuation model for share issuance/redemption fairness.
- 3 Rebalance policy hardening and safer deployment defaults.
- 4 Single-source automation topology with strict idempotency and owner-index drift resilience.
- 5 Explicit user-facing disclosure and telemetry for accounting vs realizable value divergence.

Required disclosures for users in 4626fun:

- This is an actively managed CLMM strategy, not a passive guaranteed-liquidity vault.
- Displayed NAV/accounting values can differ from immediate executable exits.
- Withdrawals may be delayed or queued under policy/market constraints.
- Oracle, keeper, and automation liveness are part of the trusted system boundary.
- Governance/manager policy changes can materially change outcomes.

18. Missing Information

Highest-value missing artifacts to finalize production-grade confidence:

- Full deployed address map per creator for vault/wrapper/strategy/charm/oracle/automation actors.
- Live onchain parameter snapshots for each deployed Charm vault and 4626 strategy weights.
- Definitive production-enabled workflow topology (which charm producer/executor path is authoritative).
- Real incident history for queue/executor retries, duplicates, and prolonged outages.
- Bundler/paymaster SLOs and CSW owner-index drift runbook used in production.
- Final user-facing valuation and redemption semantics copy (what users are actually told).